

DEPLOYMENT PLAN · 28 SEPTEMBER 2026

CrazyChat Launch Topology

How many servers CrazyChat needs to go live, what runs on each, how big they should be, and what to have ready before traffic moves.

CORE-1

DB-2

MEDIA-1

LIVE-1

Four servers in one region carry the launch

The core stack runs as the repo ships it. Call media and broadcast rendering get their own machines, and a live standby protects the one thing you cannot rebuild: the database.

CORE-1

API and primary database

CPU16 vCPU

8 dedicated cores · min 8 vCPU

Memory64 GB

min 32 GB

Disk2 × 1 TB NVMe, RAID 1

min 2 × 512 GB

Network1 Gbps

public IPv4 + private network

nginx · Caddy · go-api · PgBouncer · Postgres 16 · Redis 7 · Valhalla · nightly backup

Internet80/tcp443/tcp

DB-2

Standby and monitoring

CPU8 vCPU

min 4 vCPU

Memory32 GB

64 GB if it must run the core stack after a failover

Disk2 × 1 TB NVMe, RAID 1

holds a full copy; match CORE-1

Network1 Gbps

private network only inbound

Postgres 16 hot standby streaming from CORE-1 · Prometheus · Grafana alerting

Internetnone

MEDIA-1

Calls: SFU and TURN

CPU8 vCPU

min 4; an SFU forwards and never transcodes

Memory16 GB

min 8 GB

Disk80 GB SSD

min 40 GB

Network1 Gbps, generous traffic

bandwidth is its capacity

LiveKit calling SFU · coturn relay · local Redis for LiveKit

Internet34785349443/udp78817882/udp49152-65535/udp

LIVE-1

Go Live broadcasting

CPU16 vCPU, dedicated

min 8; 4 per live broadcast

Memory32 GB

min 16 GB

Disk80 GB SSD

segments go straight to R2

Network1 Gbps

viewers are served by Cloudflare

Go Live SFU · egress (Chrome + ffmpeg → HLS) · local Redis for Go Live

Internet7891/tcp7892/udp

Recommended48 vCPU · 144 GB RAM · 2.2 TB usable NVMe

Minimum24 vCPU · 72 GB RAM

ManagedCloudflare DNS, CDN, R2 · FCM · Expo push · MSG91 · Resend · Klipy

Start with four servers; move to the growth layout when a limit is reached

SINGLE BOX		LAUNCH • RECOMMENDED	GROWTH
Servers	1	4	9 or more
Fits	Closed beta, soft launch	Public launch. Tested envelope: 5,000 live connections per go-api process, about 50,000 accounts at 10% peak concurrency	100,000+ accounts
Kept apart	Nothing. Media and rendering share the database host	Calls, broadcasts and a standby database each get their own machine	Every tier, with the API behind a load balancer
Data lost if the main host dies	Up to 24 hours (nightly dump)	Seconds (streaming standby)	Seconds
Getting back up	New box, restore the dump, copy 9.8 GB of routing tiles	Promote DB-2 and start the core stack on it	API fails over on its own; promote the standby
Work beyond the repo	None; this is today's shape	10 config changes (S1–S10)	Those, plus a load balancer, the two replica fixes and multi-node LiveKit

Three things to settle first

✗ Sizing is planning math

The Go backend that serves all traffic has never been load-tested: every results table in `docs/PERF_BASELINE.md` is empty.

The figures here come from the Node benchmark it replaced and the repo's LiveKit notes.

Do: run `scripts/bench/run.sh` on CORE-1-class hardware and replace the planning figures.

i Host near your users

The repo points at India: routing uses `india-latest.osm.pbf`, Photon defaults to `REGION=in`, and SMS goes through MSG91. `nginx/sites/vaultchat.conf` measured a cold request from India at about 640 ms, 600 ms of it round trips to Germany.

Do: pick a Mumbai region. The topology is the same anywhere.

i Keep the hostnames

The app has `https://api.corefinite.com` compiled in (`constants/server.ts`).
A new hostname needs a new app release and leaves every installed copy talking to the old box.

Do: keep the same names, so moving servers is a DNS change.

One Go service does almost all of the work

■ **API tier**

Limited by connections and CPU

nginx 1.24
TLS for `api.`, `stream.`, `admin.`;
WebSocket upgrade

Caddy 2
Router: `/internal/*` 404, LiveKit
signalling, legal pages, rest to go-api

go-api Go 1.26
Every REST module, CC-Wire realtime,
jobs, LiveKit tokens, push

■ **Data tier**

Limited by disk I/O and memory

PgBouncer 1.23
2,000 client connections onto 40
Postgres connections

Postgres 16
Accounts, chats, E2EE ciphertext, key
bundles; migrations to 138

Redis 7
Fan-out across replicas, presence, rate
limits, job locks

■ **Media tier**

Limited by bandwidth; egress by CPU

LiveKit, calls 1.13.5
Forwards E2EE frames it cannot read;
rooms up to 200

coturn 4.6.2
TURN relay for strict NATs and
VaultBeam transfers

Go Live SFU + egress 1.13.5 · 1.14.0
Stage of 20; Chrome + ffmpeg write 4-
second HLS

■ **Helpers**

Limited by memory and disk

Valhalla pin digest
Road routing and distances; 9.8 GB of
India tiles

Photon optional
In-house address search instead of
`photon.komoot.io`

Prometheus, Grafana 2.53 · 11.1
Scrape go-api metrics; SLI board; alert
rules

Not deployed: Kafka, the Node API and its fan-out worker (legacy profile, replaced by in-process fan-out). The calling-side egress container is optional, since broadcasts use Go Live's. The external games server has no source in this repo; carry it over only if the games mini-app stays.

Messages and calls go through CORE-1; media does not

Send a message Stored and forwarded as ciphertext: X3DH and Double Ratchet for 1:1, sender keys for groups



Join a call The SFU forwards encrypted frames and never transcodes, so it needs bandwidth, not CPU

SIGNALLING

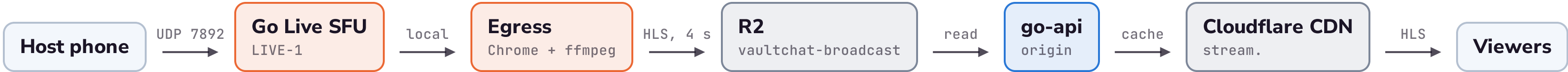


MEDIA



Behind a strict NAT the phone relays through coturn on 3478, 5349 or UDP 443. About 600 kbps per forwarded video stream; a 10-person video call is about 54 Mbps.

Go live Broadcast media is not end-to-end encrypted by design, which is what lets a CDN cache it



File bytes skip your servers; media servers report back privately

Send an attachment

go-api only signs the URL; the encrypted blob goes straight to R2



The URL is signed for `S3_PUBLIC_ENDPOINT`, which must be the R2 S3 API host. A custom domain does not check signatures, so it cannot be the gate.

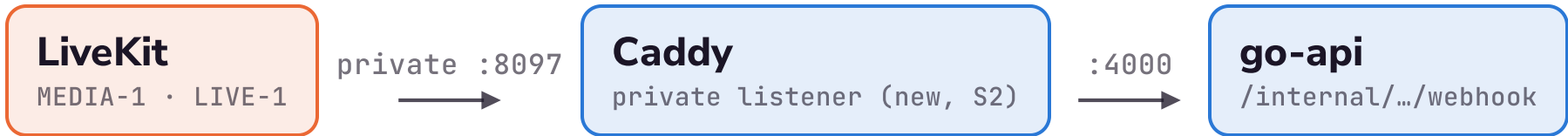
Ring a phone that is asleep

Needs the Firebase service account at `/run/secrets/firebase-sa.json`



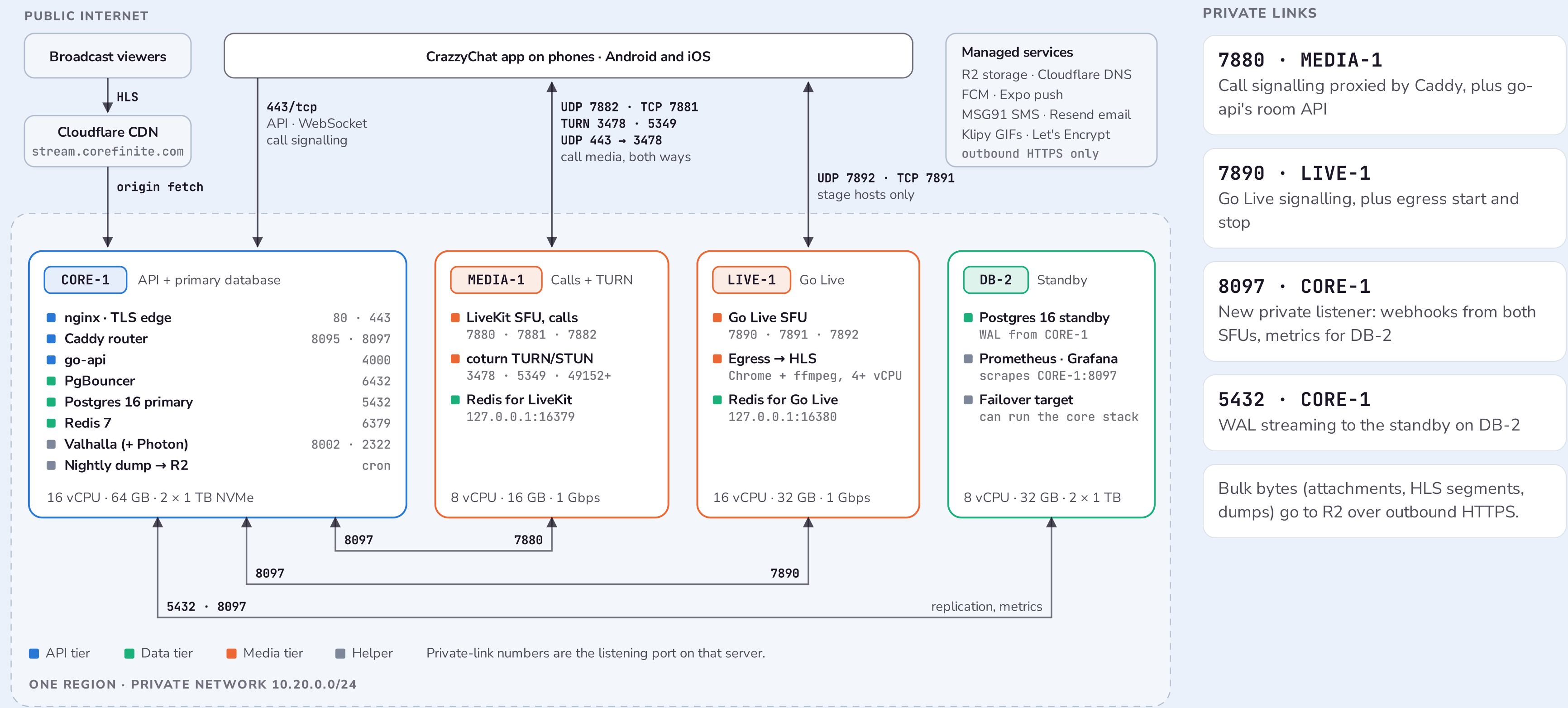
Media servers report back

Signed webhooks are the only way go-api learns a broadcast has its first segment



Without this path every broadcast stays in "starting" until a reaper cleans it up.

Only 443 reaches CORE-1; live media goes direct



Every server uses the same checkout and starts only its own services

SERVER	COMPOSE SERVICES	HOST PIECES OUTSIDE DOCKER	ITS OWN <code>./.ENV</code> HOLDS
CORE-1	All three compose files (<code>dc</code> alias): <code>postgres pgbouncer redis</code> , then <code>go-api</code> , <code>valhalla</code> , <code>caddy</code>	<code>nginx</code> , <code>certbot</code> , admin panel files, log API service, firewall units, backup cron	Sections A and B of <code>deploy/env.example</code> , plus <code>secrets/</code>
DB-2	<code>--profile monitoring</code> : <code>prometheus grafana</code> ; Postgres 16 standby from the same <code>postgres:16</code> image	Replication settings, Grafana contact point	No app secrets; a replication password
MEDIA-1	<code>redis coturn</code> , then <code>--profile sfu livekit</code> only	<code>certbot</code> for <code>turn.</code> and its deploy hook, UDP 443 → 3478 redirect, UDP buffer <code>sysctl</code>	<code>TURN_SECRET</code> , <code>TURN_EXTERNAL_IP</code> , <code>LIVEKIT_KEYS</code>
LIVE-1	<code>--profile golive</code> : <code>golive-redis golive-livekit</code> , then <code>golive-egress</code> with <code>--no-deps</code>	UDP buffer <code>sysctl</code>	<code>GOLIVE_LIVEKIT_KEYS</code> , R2 <code>S3_*</code> , <code>BROADCAST_BUCKET</code> , <code>GOLIVE_EGRESS_CPUS=12</code> , <code>GOLIVE_EGRESS_MEMORY=6g</code>

The app is CrazyChat, but the clone directory must still be named `vaultchat` on every server, so container names match the repo's scripts (`vaultchat-postgres-1`, `vaultchat-golive-egress-1`).

Each tier runs out of something different

■ go-api

Connections and delivery rate

5,000 per process

Connections held with zero failures, about 20 KB each

MEASURED · NODE

1,500 / s per process

Deliveries; half the Node knee of about 3,000/s

REPO THRESHOLD

■ Calls

Bandwidth

600 kbps per stream

Per forwarded video stream; a 10-person call is about 54 Mbps

REPO ESTIMATE

Low thousands participants

Before one LiveKit node needs a cluster

REPO ESTIMATE

■ Go Live

CPU

4 vCPU per broadcast

Egress floor; below it every broadcast stalls in "starting"

MEASURED · BENCH

20 · 70 stage · room

Publishers per broadcast and room cap; viewers are unbounded via HLS

CONFIG

■ Postgres

Disk

1.5 KB per message

Stored size with indexes; every message stays in `messages.content`

ASSUMPTION

40 connections

PgBouncer's real Postgres connections, shared by every replica

CONFIG

Measured on today's box: go-api idles at 8.7 MiB; load average 0.14 on 12 threads with 55 GB free for 18 accounts; the whole database restores in under 10 seconds. **Never measured:** the Go backend under load.

Four servers hold the launch; 100,000 users needs the growth layout

AT THE PEAK MOMENT	CLOSED BETA · 1K	LAUNCH · 10K	GROWTH · 100K	SCALE · 1M
Live connections	100	1,000	10,000	100,000
Deliveries per second	1.7	17	167	1,667
Call media through the SFU	1.1 Mbps	11 Mbps	111 Mbps	1.1 Gbps
Broadcast rendering	4 vCPU	8 vCPU	20 vCPU	80 vCPU
Postgres growth per year	8.8 GB	88 GB	876 GB	8.8 TB
Attachments held in R2	48 GB	480 GB	4.8 TB	48 TB
Layout	Launch fits; a single box also works for a closed beta	Launch fits; largest driver is LIVE-1 at 67%	Growth: connections 2×, rendering 1.7×, database 1.5× launch capacity	Growth: every driver over; measure before buying
Servers you run	4 (or 1)	4	11 APP 2 · DB 2 · Redis · GEO · MEDIA 2 · LIVE 2 · OPS	25 APP 10 · DB 2 · Redis · GEO · MEDIA 3 · LIVE 7 · OPS

Assumes 40% of accounts active daily, 10% connected at the peak moment, 40 messages per active user per day, a 3× peak hour, 3 socket deliveries per message, 3% of connected users in calls (40% video, 2.4 people each), 1, 2, 5 and 20 live broadcasts, and 4 MB of attachments per active user per day kept 30 days. Growth and scale layouts add a managed load balancer. The web version has an interactive calculator for other inputs.

Default-deny: these are the only open ports

CORE-1

443/tcp internet
API, CC-Wire WebSocket, LiveKit signalling, HLS origin, admin

80/tcp internet
Redirect to HTTPS; ACME challenges

8097/tcp private
From MEDIA-1, LIVE-1, DB-2: webhooks and metrics (S2)

5432/tcp private
From DB-2 only: replication (S8)

8443/udp optional
HTTP/3 edge from docs/HTTP3_EDGE.md

MEDIA-1

3478 · 5349 udp + tcp
STUN and TURN; TURN over TLS for networks that only pass TLS

443/udp → 3478
NAT redirect, IPv4 and IPv6, for networks that only allow 443

49152–65535/udp
TURN relay allocations

7881/tcp · 7882/udp
Call media; TCP is the fallback

7880/tcp private
From CORE-1 only: signalling and room API

LIVE-1

7891/tcp · 7892/udp
Media from stage hosts

7890/tcp private
From CORE-1 only: signalling and egress start/stop

DB-2

Nothing inbound. Grafana on 3001 through an SSH tunnel only.

All four

22/tcp admin IPs
SSH keys only, password login off, fail2ban

Docker bypasses UFW

Published ports need DOCKER-USER drops matched on --ctorigdstport. The script for this is not in the repo.

Loopback only

Caddy 8095, go-api 4000, Redis 16379 and 16380, Valhalla 8002, Photon 2322, Grafana 3001

Keep the names, move the records, copy the certificates

NAME	POINTS TO	CLOUDFLARE PROXY
api.corefinite.com	CORE-1	Off, as today
stream.corefinite.com	CORE-1	On (CDN for HLS)
admin.corefinite.com	CORE-1	On; restrict with Access or a WAF IP rule
turn.corefinite.com	MEDIA-1	Off. TURN cannot be proxied
games.corefinite.com	CORE-1	Only if the games server moves
Resend sender records	SPF, DKIM	Off

CERTIFICATE	NOTES
CORE-1 api + stream , admin	certbot with the nginx plugin; renewed by certbot.timer
MEDIA-1 turn	Issue before coturn first starts; deploy hook copies it to coturn/certs/ (65534:65534, key 640) and restarts coturn

There is no wildcard record: each name needs its own. Set TTLs to 60 seconds a day before cutover. Let's Encrypt allows 5 duplicate certificates a week, so copy /etc/letsencrypt from the old box rather than reissuing; while DNS still points at the old box, HTTP-01 cannot succeed.

Accounts, keys and secrets to have in hand

From other companies

- ☐ **REQUIRED** Access to the Cloudflare account that owns `corefinite.com`. Its credentials are not on the current box.
- ☐ **REQUIRED** R2 buckets `vaultchat-media`, `vaultchat-beam`, and a new `vaultchat-broadcast`, one S3 token per use.
- ☐ **REQUIRED** Firebase service account JSON (project `vaultchatprod01`) for push and call rings.
- ☐ **REQUIRED** MSG91 auth key and widget ID. SMS OTP is the login path.
- ☐ **REQUIRED** Resend API key and a verified sender domain.
- ☐ **REQUIRED** Google Web Client ID matching the app; Klipy production key (the test key allows 100 calls an hour).
- ☐ **REQUIRED** Play app-signing SHA-256 in `caddy/public/assetlinks.json` after the first upload.
- ☐ **RECOMMENDED** Sentry DSN; the backup job reports failures through it.

Secrets to carry over or generate

- ☐ **REQUIRED** `VAULTCHAT_MASTER_KEY` and `VAULTCHAT_LOOKUP_PEPPER`, copied from the current box. Regenerating them makes stored PII unreadable, with no recovery.
- ☐ **REQUIRED** `JWT_SECRET`, 32+ random bytes. go-api refuses to start without it; changing it signs everyone out.
- ☐ **REQUIRED** `INVITE_SECRET`, `ADMIN_KEY`, `INTERNAL_EMIT_KEY`, `DB_PASS`.
- ☐ **REQUIRED** `TURN_SECRET`, identical in coturn and go-api.
- ☐ **REQUIRED** Two different LiveKit key pairs, one for calls and one for Go Live; each key ID pasted into its YAML.
- ☐ **REQUIRED** The backup passphrase (`.offsite-key`) in a password manager, off the servers.
- ☐ **REQUIRED** SSH keys for each admin; password login off.

Hosts, data and the app

Every server

- ☐ Ubuntu 24.04 LTS, Docker Engine with Compose v2
- ☐ Time sync on; TURN credentials and JWTs are time-stamped
- ☐ `daemon.json`: live-restore, log rotation 10 MB × 3
- ☐ UFW default deny plus `DOCKER-USER` drop rules
- ☐ fail2ban and unattended security upgrades
- ☐ Pinned images: LiveKit v1.13.5, egress v1.14.0, Valhalla by digest

Per server

- ☐ CORE-1: nginx, certbot, the vhosts, and `vaultchat-stream-proxy.conf` (not in the repo)
- ☐ MEDIA-1 and LIVE-1: `rmem_max` and `wmem_max` = 5000000
- ☐ MEDIA-1: UDP 443 → 3478 redirect as a systemd unit
- ☐ MEDIA-1: certbot deploy hook for coturn's certificate
- ☐ LIVE-1: egress CPUs ≥ 4 (12 on 16 vCPU), 2 GB per 4 vCPU

Data and the app

- ☐ Valhalla India tiles (9.8 GB), copied rather than rebuilt
- ☐ Legal pages at `/privacy`, `/terms`, `/delete-account`
- ☐ If migrating: a fresh dump, restore-tested with row counts compared
- ☐ `npm run prod:precheck` passes; `ALLOW_DEV_OTP` unset
- ☐ `scripts/bench/run.sh` results entered in the calculator
- ☐ Two-phone smoke test (`docs/GO_LIVE_SMOKE_TEST.md`) passes

Hosts, database and schema before any API binary

01	Provision all four One region, one private network. CORE-1 .11 · DB-2 .12 · MEDIA-1 .21 · LIVE-1 .31 on 10.20.0.0/24	ip -4 addr show # public and private address on each
02	Prepare each host all four Docker with Compose v2, fail2ban, firewall from the port table; UDP buffers on the media hosts	curl -fsSL https://get.docker.com sh && docker compose version
03	Check out and configure CORE-1 Clean clone into <code>~/vaultchat</code>, box compose file, both env files, <code>secrets/</code>; apply S1, S2, S4, S5, S7	awk '/SECTION A ---/,/END SECTION A/' deploy/env.example > .env
04	Database first CORE-1 Postgres, PgBouncer and Redis up and ready	dc up -d postgres pgbouncer redis
05	Schema before the API CORE-1 Ledger loop from <code>deploy/README.md</code> §3.2, or restore the dump and compare the ledger	docker exec vaultchat-postgres-1 pg_isready -U vaultchat
06	Start the standby DB-2 Replication role and <code>pg_hba.conf</code> line on CORE-1; clone with the same <code>postgres:16</code> image	pg_basebackup -h 10.20.0.11 -U replicator -D /var/lib/postgresql/data -R -X stream -P
07	The API CORE-1 Always build with <code>--pull</code>; start Valhalla once the tiles are in place	dc build --pull go-api && dc up -d --no-deps go-api

Media, router and edge last, then measure

08	Call media MEDIA-1 Certificate before coturn ever starts; edit <code>livekit.yaml</code> (S3) first	<pre>docker compose -f docker-compose.yml -f docker-compose.prod.yml --profile sfu up -d livekit</pre>
09	Broadcasting LIVE-1 Egress with <code>--no-deps</code> , or compose also starts MinIO; check there is no "not enough cpu" line	<pre>... --profile golive up -d --no-deps golive-egress</pre>
10	Router last CORE-1 Validate the Caddyfile; a router started before its upstreams answers 502 to everything	<pre>dc up -d caddy && curl -s localhost:8095/health</pre>
11	Edge and TLS CORE-1 Certificates first or <code>nginx -t</code> fails; copy the vhosts and the stream proxy include	<pre>sudo nginx -t && sudo systemctl reload nginx</pre>
12	Backups and monitoring CORE-1 DB-2 Nightly encrypted dump to R2 on CORE-1; Prometheus and Grafana on DB-2 with a contact point	<pre>... --profile monitoring up -d prometheus grafana</pre>
13	Verify CORE-1 <code>/health</code> , <code>/ready</code> , <code>/build</code> fingerprint, sfu-token 401, <code>/golive/health</code> ; then two phones	<pre>curl -s localhost:8095/build # equals scripts/fingerprint-go.sh</pre>
14	Measure CORE-1 Before real data lands, or on a clone. It seeds test users; never aim it at the live database	<pre>bash scripts/bench/run.sh</pre>

Ten edits the four-server split needs; none are in the repo yet

#	CHANGE	WHERE	WITHOUT IT
S1	Caddy <code>/livekit</code> → <code>10.20.0.21:7880</code> , <code>/golive-livekit</code> → <code>10.20.0.31:7890</code>	<code>caddy/Caddyfile</code>	Every call and broadcast join fails
S2	Private Caddy site on <code>10.20.0.11:8097</code> forwarding only the two webhooks and <code>/internal/metrics</code>	Caddyfile, box compose	Broadcasts never leave "starting"; no metrics
S3	Both LiveKit YAMLs: <code>node_ip</code> = that server's public IP, webhook URL → 8097, literal <code>api_key</code>	<code>livekit/*.yaml</code>	Clients get the old box's IP; webhooks lost
S4	<code>LIVEKIT_HTTP_URL</code> and <code>GOLIVE_LIVEKIT_HTTP_URL</code> to the private IPs; <code>TURN_HOST</code>	CORE-1 <code>.env</code>	Room and egress calls detour via the edge
S5	Broadcast segments on R2: egress <code>S3_*</code> on LIVE-1 and <code>BROADCAST_S3_*</code> on CORE-1 name the same bucket	LIVE-1, CORE-1 <code>.env</code>	Every segment 404s
S6	<code>TURN_EXTERNAL_IP</code> = MEDIA-1's IP, <code>turn.</code> certificate and hook, UDP 443 redirect	MEDIA-1	Relayed calls fail
S7	Pin the compose network to <code>172.28.0.0/16</code> , or set <code>TRUSTED_PROXIES</code>	box compose	One user can lock everyone out of OTP
S8	Postgres published on <code>10.20.0.11:5432</code> only; replication role and <code>pg_hba.conf</code>	box compose, Postgres	No standby
S9	Prometheus targets 8097, loads <code>alerts.yaml</code> , and has a contact point	<code>monitoring/</code>	Alerts notify nobody (today's state)
S10	Drop go-api's <code>14000</code> publish; land C1 and C2 before a second replica	box compose, Caddyfile, Go	Scaling blocked; duplicate daily pushes

Four repo defaults that break a fresh server

env.example breaks Go Live playback

It puts R2 in `./.env` as `S3_*`. Compose also feeds those into both egress configs, so segments go to R2 while `BROADCAST_S3_*` still reads MinIO. Point both halves at one store (S5).

`deploy/env.example` • `docker-compose.yml` • `internal/storage/storage.go`

`host.docker.internal` does not resolve in go-api

`deploy/env.example` suggests `LIVEKIT_HTTP_URL=http://host.docker.internal:7880`, but only Caddy and egress get the host-gateway mapping. Use private IPs (S4).

`docker-compose.yml` `extra_hosts` • `internal/livekit/token.go`

The old IP is baked into three defaults

`TURN_EXTERNAL_IP` in compose, `node_ip` in both LiveKit YAMLS, and the `S3_PUBLIC_ENDPOINT` fallback all say 65.21.229.167. Set each explicitly.

`docker-compose.yml` • `livekit/livekit.yaml` • `livekit/golive.yaml`

The stream vhost includes a missing file

`vaultchat-stream.conf` includes `/etc/nginx/vaultchat-stream-proxy.conf`, which exists only on the old box. Copy it or `nginx -t` fails.

`nginx/sites/vaultchat-stream.conf` • `docs/SERVER_INVENTORY.md` §5

Most of these fail silently: the stack looks healthy while a feature is dead

COMPOSE AND CONFIGURATION

Profiles hide the media stack

A plain `up -d` starts 8 of 17 containers; start media services by name

Never `--remove-orphans` or `down -v`

The first deletes live media services; the second deletes volumes

Two env files

`./.env` feeds compose, `vaultchat-backend/.env` the container

The third compose file is load-bearing

Without it, attachments land in a MinIO no client can reach

LiveKit YAML has no `${VAR}` expansion

A placeholder `api_key` rejects every webhook

NETWORK, TLS AND MEDIA

UFW does not cover Docker ports

Use `DOCKER-USER` drops on `--ctorigdstport`

Egress below 4 CPUs says "ready"

Then never produces a playlist

coturn needs its cert before first start

Otherwise Docker mounts empty directories and TLS is off

LiveKit needs Redis on host loopback

Without `127.0.0.1:16379` every join returns HTTP 500

Five certificates a week

Iterated rebuilds that reissue lose HTTPS for seven days

DATA AND SECRETS

Two keys can never be regenerated

Master key and lookup pepper decrypt and index existing PII

Off-site backups failed for a week

A revoked R2 token; alert on backup age, not exit code

The backup key lived next to the backups

Store `.offsite-key` off the servers first

Build with `--pull`

The Go base image floats; prod once missed three stdlib fixes

Dev OTP must stay off

`ALLOW_DEV_OTP` logs live login codes

The standby turns a lost day of data into lost seconds

IF THIS HAPPENS	SINGLE BOX TODAY	WITH THIS PLAN
A disk fails	RAID 1 carries on	Same
CORE-1 is lost	Restore last night's dump: up to 24 h of data gone, hours to rebuild	Promote DB-2 (seconds of data), start the core stack on it
Bad migration or mass delete	Nightly dump only	Nightly dump; add WAL archiving to R2 for point-in-time restore
MEDIA-1 or LIVE-1 lost	Everything is down together	Only that feature stops; rebuild from the repo in minutes
Whole region down	Restore from R2 elsewhere	Same

BACK UP	HOW
Postgres	Nightly <code>pg_dump</code> , AES-256, 14 days in R2; standby on DB-2
Env files, box compose, <code>secrets/</code> , LiveKit YAMLs	Encrypted, in a password manager or vault
<code>.offsite-key</code>	Password manager; without it every dump is unreadable
Certs, vhosts, firewall scripts, units	A private ops repository
Valhalla tiles	A copy on DB-2
R2 objects	Held only by Cloudflare; decide if that is acceptable

Restoring today's database takes under ten seconds. What limits recovery is keys, credentials and how old the newest off-site copy is.

Alert rules exist; today they notify nobody

SIGNAL	FROM	STATUS
API down, crash loop, DB pool exhausted or cancelling, unbounded goroutines or heap	monitoring/alerts.yml	RULES WRITTEN
A person gets told: Grafana contact point or Alertmanager	DB-2 (S9)	MISSING
https://api.corefinite.com/health and /golive/health checked from outside	External uptime checker	TO ADD
Newest off-site dump older than 26 hours	Backup job plus an age check	TO ADD
Certificates expiring within 20 days, turn. included	Uptime checker or cron probe	TO ADD
Host CPU, memory, disk and network on all four servers	node_exporter or the Beszel agent	TO ADD
Latency and connection-ceiling alerts	First bench run	DRAFTED, COMMENTED OUT

Moving off the current box is a short maintenance window

01 Copy what cannot be recreated

Env files, box compose, `secrets/`, `.offsite-key`, RLS passwords, `/etc/letsencrypt`, real LiveKit YAMLS, admin files, tiles

02 Confirm backups upload

They failed from 6 September on a revoked R2 token. Take a fresh dump by hand first

03 Build alongside

Bring-up steps 1–14 with restored data. Test with `curl --resolve` and a staging app build

04 Lower DNS TTLs

`api`, `stream`, `admin`, `turn` to 60 seconds a day ahead

05 Maintenance window

Stop go-api on the old box, final dumps, restore on CORE-1, compare ledger and row counts, re-seed DB-2

06 Move objects

Mirror MinIO's 5.8 GB of broadcast segments and 521 MB of legacy media to R2, or drop old replays

07 Switch DNS

`api`, `stream`, `admin` to CORE-1; `turn` to MEDIA-1; update proxied origins

08 Verify

Step 13 checks, `/build` matches the deployed tree, two-phone smoke test

09 Keep, then wipe

Old box stopped for 14 days as rollback; then destroy it and its three loose credential files

What it takes to go live

01 Approve the four-server plan and the region

CORE-1, DB-2, MEDIA-1, LIVE-1 in one region near users; a Mumbai region if most users are in India.

02 Run the Go benchmark

`scripts/bench/run.sh` on CORE-1-class hardware; replace the planning figures before buying more.

03 Collect credentials and keys

Work through the prerequisites, starting with the master key, pepper and backup passphrase.

04 Land S1–S10 and fix the four repo traps

Config for the split, plus the `env.example`, `host-gateway`, `old-IP` and `nginx` include defaults.

05 Bring up and verify

Fourteen steps in order, then the two-phone smoke test on the new servers.

06 Schedule the cutover

Lower TTLs, one maintenance window, keep the old box for 14 days.

Sources: `deploy/README.md`, `deploy/env.example`, `docker-compose*.yaml`, `docs/SERVER_INVENTORY.md`, `docs/REBUILD_RISKS.md`, `docs/SECOND_REPLICA_READINESS.md`, `docs/SFU_DEPLOY.md`, `docs/GOLIVE_DEPLOY.md`, `docs/BACKUP_VERIFICATION.md`, `docs/PERF_BASELINE.md`, `vaultchat-backend/loadtest/REPORT.md`. Full page: `docs/LAUNCH_TOPOLOGY.html`.